Tema 6 · Restricciones
Búsqueda con espacios estructurados

Restricciones: modelado, búsqueda y propagación. Material de estudio del módulo «Agentes basados en Búsqueda», URJC, curso 2026/27. El nombre completo de la asignatura no figura explícitamente en los archivos y no se presupone.

Modelado CSP, búsqueda, propagación y MAC, con los ocho ejercicios integrados y sus procedimientos explicados paso a paso.

Cómo leer las correcciones. Los recuadros «Precisión sobre la fuente» señalan erratas o convenciones que afectan al razonamiento. Las animaciones se reconstruyen en estados y tablas a partir de sus objetos y secuencias; no se han reproducido en PowerPoint.

Índice navegable

1. De los estados opacos a los CSP 2

2. Modelar y representar restricciones 3

3. Convertir restricciones no binarias 8

4. Dos espacios de búsqueda 9

5. Vuelta atrás y decisiones de búsqueda 11

6. Propagación y comprobación hacia delante 14

7. Consistencia de arco y AC-3 18

8. Ejecutar consistencia de arco en problemas nuevos 22

9. Límites de la propagación y k-consistencia 24

10. MAC: búsqueda y propagación combinadas 26

11. Modelar y ejecutar una agenda de visitas 28

12. Extensiones y preparación de la evaluación 30

13. Fuentes y alcance de las reconstrucciones 32

1. De los estados opacos a los CSP

1.1. Qué aporta la estructura del estado

En los temas anteriores, un estado podía tratarse como un símbolo: s_0 era el inicial; expandir(s_7) devolvía, por ejemplo, {s_8,s_{10},s_{12},s_{27},s_{112}}; meta?(s_{112}) era verdadero; c(s_7,s_{112})=5 y la función heurística de la diapositiva devolvía 235 para s_7. El buscador utilizaba esas funciones sin conocer propiedades como «A está sobre B» o «el agente está en Radovac».

Esta representación monolítica separa el algoritmo del dominio, pero puede hacer complejas las funciones de sucesión y difícil mantener el conocimiento. Un estado estructurado expone propiedades mediante variables. Las reglas se expresan sobre ellas y permiten descartar valores antes de construir un estado completo. Sudoku, horarios docentes y planificación de producción son ejemplos de esta idea.

1.2. Definición y notación

Un **problema de satisfacción de restricciones**, o CSP (Constraint Satisfaction Problem), es una tripleta (X,D,R). X={x_1,…,x_n} es un conjunto finito de variables. D asigna a cada variable x_i un dominio finito D_i de valores permitidos. R={R_1,…,R_k} es un conjunto finito de restricciones. Cada restricción establece qué combinaciones de valores admite sobre un subconjunto de variables, denominado su ámbito.

Si R_j afecta a x_a,x_b,x_c, se representa como una relación $R_j \subseteq D_a \times D_b \times D_c$. Una tupla pertenece a la relación si satisface el predicado. Por ejemplo, con dominios {1,2,3}, la restricción A<B puede escribirse por una fórmula o de forma extensiva como {(1,2),(1,3),(2,3)}. La fórmula suele ser compacta; la lista permite comprobar pertenencia directamente.

Precisión sobre la fuente (diapositivas 5 y 10). D devuelve conjuntos de valores, no un único valor. Formalmente puede escribirse D:X→P(V), donde P(V) es el conjunto de subconjuntos de V, con D(x_i) finito. Una asignación σ tiene un subconjunto de X como dominio y cumple σ(x_i)∈D_i. Las notaciones D:X→V y σ:X→D(X) de las diapositivas deben leerse con esta intención.

Una restricción es **unaria** si afecta a una variable, **binaria** si afecta a dos y **k-aria** si afecta a k. En un CSP binario todas las restricciones tienen aridad como máximo dos. Una restricción global reúne una condición sobre muchas variables, como todos-diferentes. «Global» no significa que obligatoriamente intervengan todas las variables del CSP.

1.3. Asignación, consistencia y solución

Una asignación parcial σ fija valores para algunas variables: {x_1=2,x_3=1}. Es completa cuando asigna todas las variables. Una asignación completa es una **solución** si sus valores pertenecen a los dominios y satisfacen todas las restricciones. Completa no equivale a correcta: se puede rellenar todo el tablero y mantener conflictos.

En búsqueda con asignaciones parciales, una asignación es consistente si no viola ninguna restricción cuyo ámbito esté ya completamente asignado. Para una restricción todavía incompleta no se evalúa una tupla con componentes inexistentes. La propagación puede detectar además que una asignación parcial no admite extensión, aunque sus

restricciones ya evaluables se cumplan.

En las cuatro reinas, $\{x_1=2,x_2=4,x_3=3\}$ cumple la restricción entre filas 1 y 3, pero no es consistente globalmente: $x_2=4$ y $x_3=3$ se amenazan en diagonal. $\{1,2,3,4\}$ es completa y viola la restricción entre filas 1 y 3. $\{2,4,1,3\}$ es completa y sí es solución. El primer ejemplo de la diapositiva 10 solo afirma cumplimiento de una restricción concreta.

Aplicación al test 1.1. Solo (b) es verdadera: el operador asigna una variable no asignada. (a) confunde asignación parcial con completa; (c) omite que la meta debe ser completa; (d) no describe ninguna propiedad exigida de un CSP y el coste docente es cero; (e) corresponde al enfoque de asignaciones completas y conflicto mínimo.

2. Modelar y representar restricciones

2.1. Las n reinas: de la geometría a las relaciones

Hay que situar n reinas en un tablero n×n sin que se ataquen. Elegimos una variable x_i por fila; su valor es la columna de la reina de esa fila. Así, $X=\{x_1,\ldots,x_n\}$ y $D_i=\{1,\ldots,n\}$. La propia representación garantiza una reina por fila. Para cada par $i<j$ se exige:

Restricción entre dos reinas: $x_i \neq x_j$ y $|x_i-x_j| \neq |i-j|$.
La primera condición impide compartir columna. La segunda impide compartir diagonal: en una diagonal, la distancia horizontal coincide con la vertical.

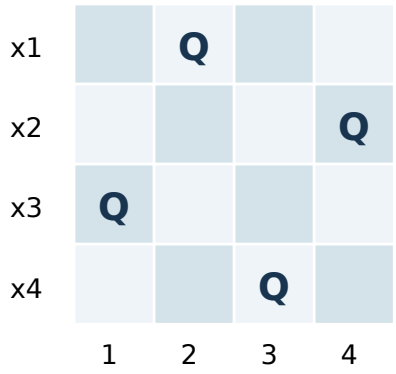


Figura 1. Una solución de cuatro reinas, (2,4,1,3). Reconstrucción de la diapositiva 6; Q identifica la reina de cada fila.

Para cuatro reinas, también se podría usar una única restricción de aridad cuatro con las dos tuplas $\{(2,4,1,3),(3,1,4,2)\}$. Eso describe el mismo conjunto de soluciones, pero exige conocerlas de antemano. La formulación por pares expresa reglas locales y permite encontrarlas mediante búsqueda.

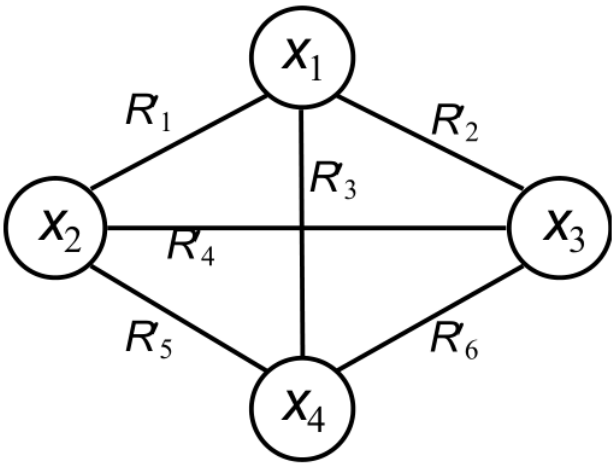


Figura 2. Grafo de las cuatro reinas: reproducción equivalente del ejercicio 5 (p. 4) y la diapositiva 7. Cada arista expresa ausencia de amenaza entre las dos filas.

Restricción	Variables	Distancia de filas
R'_1	x_1,x_2	1
R'_2	x_1,x_3	2
R'_3	x_1,x_4	3
R'_4	x_2,x_3	1
R'_5	x_2,x_4	2
R'_6	x_3,x_4	1

El ejercicio 5 agrupa las relaciones por distancia. En cada par (a,b), a y b son columnas, no números de fila. Las listas verificadas son:

$\mathbf{R^{d1}}$ = {(1,3),(1,4),(2,4),(3,1),(4,1),(4,2)}.
$\mathbf{R^{d2}}$ = {(1,2),(1,4),(2,1),(2,3),(3,2),(3,4),(4,1),(4,3)}.
$\mathbf{R^{d3}}$ = {(1,2),(1,3),(2,1),(2,3),(2,4),(3,1),(3,2),(3,4),(4,2),(4,3)}.

$R'_1=R'_4=R'_6=R^{d1}$; $R'_2=R'_5=R^{d2}$; $R'_3=R^{d3}$. Por ejemplo, (2,4) se admite para filas contiguas, pero se prohíbe a distancia dos porque sería diagonal. Esta distinción evita aplicar una tabla correcta al par equivocado.

2.2. Grafos, hipergrafos y todos-diferentes

Un CSP binario se representa mediante un nodo por variable y una arista por pareja relacionada. Es conveniente etiquetar las aristas con sus predicados; dos grafos con la misma forma pueden expresar problemas distintos. Para propagación, cada arista se considera en dos direcciones. La dirección indica qué dominio se revisa, aunque la relación matemática no sea simétrica.

En un hipergrafo, una restricción de aridad k conecta k variables. Puede dibujarse mediante un punto o pequeño nodo de restricción unido a todas ellas. Ese punto organiza el dibujo; solo se convierte en una variable del CSP cuando se aplica una codificación como la de variables ocultas (§3).

Todos-diferentes significa $\forall i<j,\ x_i \neq x_j$. La cadena $x_1 \neq x_2 \neq x_3$ no basta si se interpreta literalmente como desigualdades consecutivas: (1,2,1) las cumple, aunque repite 1. En las diapositivas y el Sudoku, esa cadena se utiliza como abreviatura informal de desigualdad entre todos los pares.

2.3. Coloración de Australia (ejercicio 2)

Se pide colorear los siete territorios con rojo, verde y amarillo sin igualar colores en territorios adyacentes. X={WA,NT,SA,Q,NSW,V,T}. Cada dominio es {r,v,a}. Para cada frontera del mapa se impone desigualdad; no se exige que territorios sin frontera tengan colores diferentes.

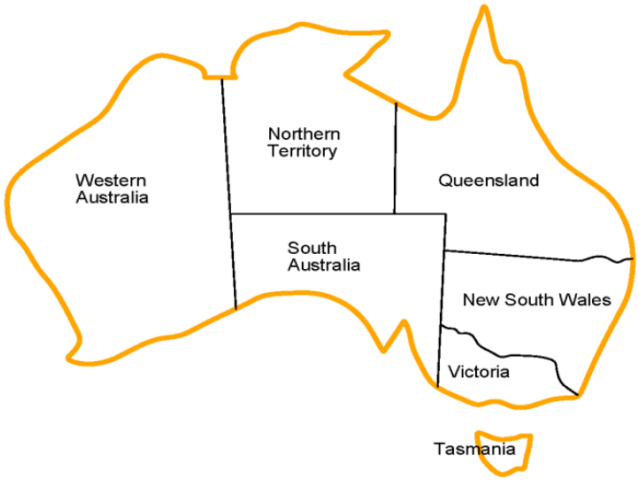


Figura 3. Mapa original del ejercicio 2 (p. 2). Tasmania aparece separada del continente.

Restricciones	Parejas con frontera
R_1 a R_3	WA–NT; WA–SA; NT–Q
R_4 a R_6	NT–SA; Q–SA; Q–NSW
R_7 a R_9	SA–NSW; SA–V; NSW–V

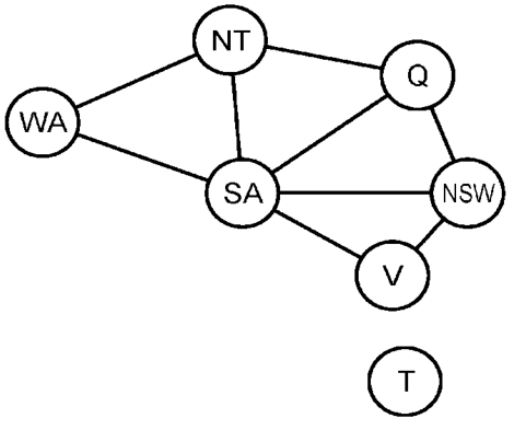


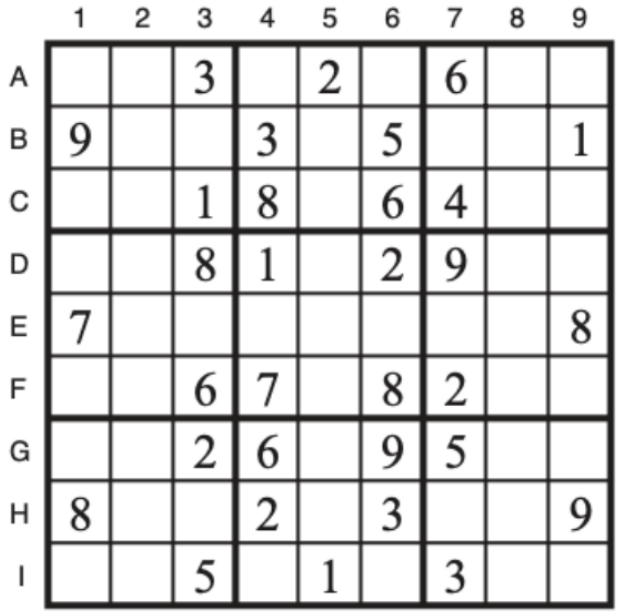
Figura 4. Grafo original del ejercicio 2. El nodo T aislado conserva su dominio completo.

Una solución es {WA=a,NT=r,SA=v,Q=a,NSW=r,V=a,T=r}. Se comprueba leyendo las nueve aristas: sus extremos tienen colores distintos. T puede usar cualquiera de los tres

colores. Un error típico es conectar territorios por proximidad visual o imponer todos-diferentes sobre los siete: ambas cosas cambiarían el problema. Si se añade una nueva frontera, hay que añadir su desigualdad y volver a verificar la solución.

2.4. Sudoku como CSP global (ejercicio 3)

El tablero de 9×9 tiene 81 variables, A1,…,A9,B1,…,I9. Una casilla vacía tiene dominio {1,…,9}; una casilla que contiene la pista k tiene dominio {k}. Las pistas forman parte del problema, no son valores que el buscador pueda cambiar. Hay 27 restricciones todos-diferentes: nueve filas, nueve columnas y nueve bloques de 3×3.



	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

Figura 5. Tablero inicial del ejercicio 3 (p. 2), conservado para identificar las pistas y los bloques.

Por ejemplo, la primera fila usa todos-diferentes(A1,…,A9); la primera columna, todos-diferentes(A1,…,I1); y el bloque superior izquierdo, todos-diferentes(A1,A2,A3,B1,B2,B3,C1,C2,C3). La figura fija, entre otras pistas, A3=3, A5=2 y A7=6. El patrón se repite en las nueve filas, columnas y bloques, sin añadir restricciones sobre diagonales.

Errata del solucionario (p. 3). La lista del dominio de un Sudoku vacío incluye 0, en contradicción con el enunciado y con el texto de la propia solución. El dominio correcto es {1,…,9}. Aunque la relación global escrita sobre {1,…,9} ya impediría usar 0, incluirlo en los dominios es incoherente y añade candidatos inútiles.

Cada todos-diferentes puede descomponerse en desigualdades binarias entre todos sus pares. Las soluciones se conservan, pero un propagador especializado para la restricción global puede eliminar más valores que la consistencia de arco aplicada separadamente a esas desigualdades. Esta distinción es una explicación auxiliar: igualdad de soluciones no implica igualdad de fuerza de propagación.

2.5. Criptoaritmética: TWO + TWO = FOUR

Cada letra representa un dígito y las letras diferentes deben tener dígitos distintos. Para expresar la suma por columnas se introducen tres acarreos C_1,C_2,C_3. El conjunto de variables es $\{T,W,O,R,U,F,C_1,C_2,C_3\}$. La fuente usa $D_T=D_W=D_O=D_R=D_U=\{1,\ldots,9\}$ y $D_F=D_{C_1}=D_{C_2}=D_{C_3}=\{0,1\}$.

Columna / relación	Condición	Interpretación
Unidades, R_1	$O+O=R+10C_1$	R es la unidad del resultado; C_1 pasa a decenas.
Decenas, R_2	$W+W+C_1=U+10C_2$	Se suma el acarreo anterior.
Centenas, R_3	$T+T+C_2=O+10C_3$	O también aparece en el resultado.
Millares, R_4	$F=C_3$	El último acarreo forma el primer dígito.
Global, R_5	todos-diferentes(T,W,O,R,U,F)	No incluye los acarreos.

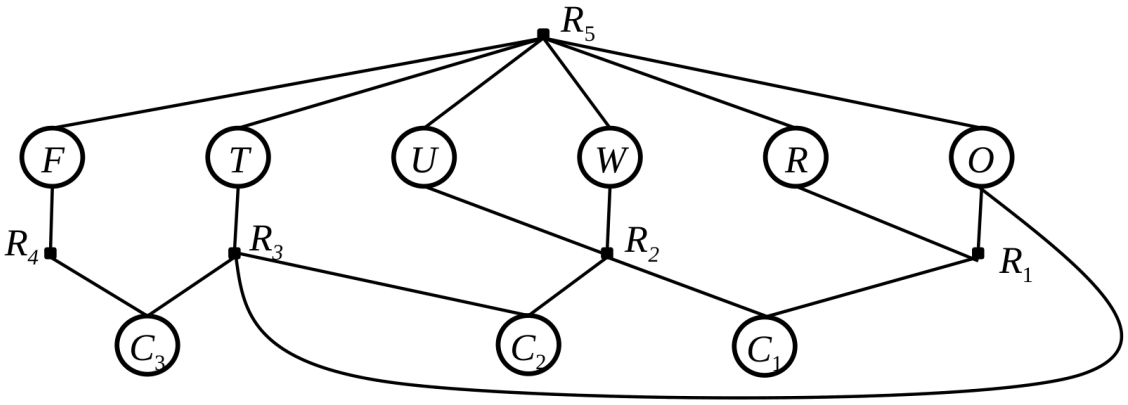


Figura 6. Hipergrafo original de la diapositiva 9. R_1 y R_3 comparten O; R_5 conecta las seis letras y no los acarreos.

Los acarreos valen 0 o 1 porque sumar dos dígitos y un acarreo produce como máximo 19. Sin embargo, las condiciones escritas no obligan a F=1: 132+132=0264 satisface literalmente los dominios y las cinco restricciones, con T=1,W=3,O=2,R=4,U=6,F=0 y todos los acarreos a 0. Es una suma de tres cifras, no un FOUR de cuatro cifras. Para imponer la interpretación habitual hay que añadir $F\neq 0$, de modo que $F=C_3=1$.

Convención particular de la diapositiva 8. La criptoaritmética habitual permite 0 en letras que no encabezan un número: W,O,R,U tendrían $\{0,\ldots,9\}$, T tendría $\{1,\ldots,9\}$ y $F=\{1\}$. La fuente excluye 0 de las cinco primeras letras y deja F inicialmente en $\{0,1\}$. Son modelos diferentes; aquí se conserva el dominio docente y se señala la condición de cuatro cifras. No debe extenderse todos-diferentes a los acarreos, que sí pueden coincidir entre sí o con letras.

Como comprobación añadida, 734+734=1468 respeta ambas formulaciones: T=7,W=3,O=4,R=8,U=6,F=1. Unidades: 4+4=8, $C_1=0$. Decenas: 3+3+0=6, $C_2=0$. Centenas: 7+7+0=4+10, $C_3=1$. Finalmente F=1. El ejemplo muestra por qué hacen falta variables auxiliares y cómo se verifica una solución por columnas.

3. Convertir restricciones no binarias

3.1. Codificación de variables ocultas

Una relación k-aria no puede sustituirse sin más por todas sus proyecciones binarias: esas proyecciones podrían admitir combinaciones que la relación original prohíbe. El procedimiento general del ejercicio 4 introduce una variable auxiliar v_R cuyo dominio contiene exactamente las tuplas permitidas por R. Si R afecta a $(x_1,\ldots,x_k)$, se añaden k restricciones binarias $x_i=v_R[i]$. Aquí v[i] significa la componente i de la tupla elegida, con índices desde 1.

La equivalencia se demuestra en dos direcciones. Dada una solución original, el valor de v_R se fija a la tupla que forman sus variables: esa tupla pertenece a R y cumple todas las igualdades de proyección. Dada una solución de la codificación, las igualdades obligan a que las variables originales formen la tupla de v_R, que pertenece a su dominio R; por tanto satisfacen la restricción original. La equivalencia se entiende al proyectar sobre las variables originales, pues el CSP convertido tiene variables adicionales.

3.2. Las seis variables booleanas (ejercicio 4)

$X=\{x_1,\ldots,x_6\}$, con dominios $\{0,1\}$. Las restricciones son $R_1(x_1,x_2,x_6)$: $x_1+x_2+x_6=1$; $R_2(x_1,x_3,x_4)$: $x_1-x_3+x_4=1$; $R_3(x_4,x_5,x_6)$: $x_4+x_5-x_6\geq 1$; $R_4(x_2,x_5,x_6)$: $x_2+x_5-x_6=0$. Cada restricción ternaria admite solo algunas de las ocho tuplas booleanas posibles.

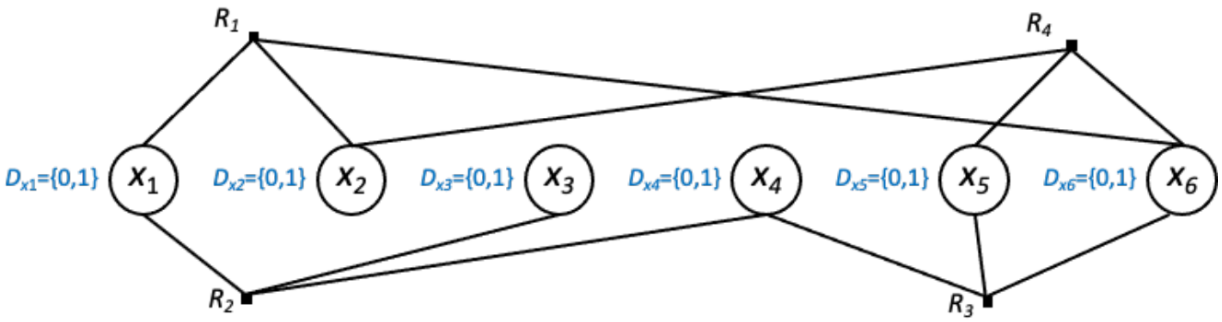


Figura 7. Hipergrafo original del ejercicio 4 (p. 4). Cada punto R_i agrupa exactamente tres variables.

Auxiliar y orden de componentes	Dominio de tuplas permitido
$v_1=(x_1,x_2,x_6)$	{(0,0,1),(0,1,0),(1,0,0)}
$v_2=(x_1,x_3,x_4)$	{(0,0,1),(1,0,0),(1,1,1)}
$v_3=(x_4,x_5,x_6)$	{(0,1,0),(1,0,0),(1,1,0),(1,1,1)}
$v_4=(x_2,x_5,x_6)$	{(0,0,0),(0,1,1),(1,0,1)}

Por ejemplo, para R_3, (1,1,1) se admite porque $1+1-1=1\geq 1$; (0,1,1) se rechaza porque da 0. Para R_2, (1,1,1) también se admite, ahora porque $1-1+1=1$. No se pueden copiar los dominios entre auxiliares aunque todas las variables originales sean booleanas.

Auxiliar	Tres restricciones binarias
v_1	$x_1=v_1[1]$; $x_2=v_1[2]$; $x_6=v_1[3]$
v_2	$x_1=v_2[1]$; $x_3=v_2[2]$; $x_4=v_2[3]$
v_3	$x_4=v_3[1]$; $x_5=v_3[2]$; $x_6=v_3[3]$
v_4	$x_2=v_4[1]$; $x_5=v_4[2]$; $x_6=v_4[3]$

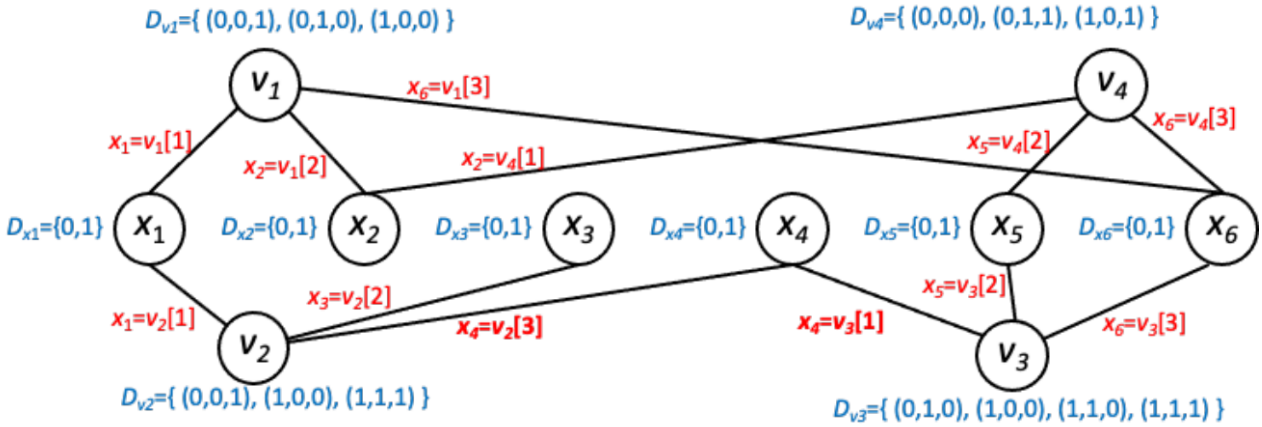


Figura 8. Codificación binaria del solucionario (p. 4): diez variables y doce restricciones de proyección. La tabla anterior permite leer las etiquetas sin depender del tamaño del gráfico.

Se conservan los dominios de las seis variables originales y se sustituyen las cuatro restricciones ternarias por estas doce binarias. Como verificación independiente, las soluciones originales son (0,0,0,1,1,1) y (1,0,1,1,0,0), en orden $x_1,\ldots,x_6$. En la primera, las tuplas auxiliares son $v_1=(0,0,1)$, $v_2=(0,0,1)$, $v_3=(1,1,1)$, $v_4=(0,1,1)$. Todas figuran en sus dominios.

El coste de la transformación está en los dominios auxiliares: una restricción sobre k variables de tamaño d puede admitir hasta d^k tuplas. Convertir a binario permite reutilizar algoritmos binarios, pero no garantiza una representación pequeña ni una ejecución más rápida.

4. Dos espacios de búsqueda

4.1. Asignaciones completas y conflicto mínimo

En este enfoque cada estado asigna todas las variables, aunque viole restricciones. Un operador cambia el valor de una variable; la meta es una asignación sin violaciones. El coste del camino se toma como cero porque interesa la asignación final, no cuántos cambios hicieron falta. La heurística de conflicto mínimo prefiere sucesores con menor número de restricciones violadas.

Para cuatro reinas se cuentan pares de reinas que se amenazan, cada par una sola vez. El estado de la diapositiva 12 es (1,4,1,4), con tres pares en conflicto: filas 1–3, 1–4 y 2–4. Cada celda de la tabla siguiente muestra los conflictos totales que resultarían al mover solo la reina de esa fila a esa columna.

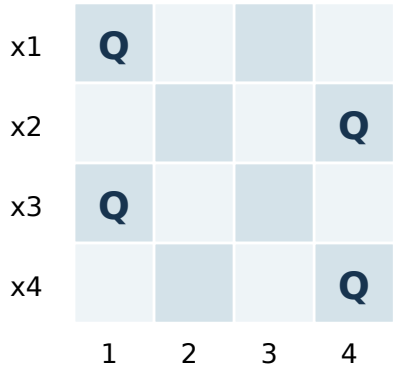


Figura 9. Estado inicial del ejemplo de conflicto mínimo, reconstruido de la diapositiva 12.

Fila que se modifica	Col. 1	Col. 2	Col. 3	Col. 4
1	3	1	3	3
2	4	5	2	3
3	3	2	5	4
4	3	3	1	3

Cambiar x_2 a 3 produce (1,3,1,4) y deja dos conflictos: 1–3 y 1–4. Cambiar x_1 a 2 produce (2,4,1,4) y deja solo el conflicto 2–4. Si después se cambia x_4 a 3 se obtiene (2,4,1,3), con cero conflictos. La tabla reproduce los 16 valores de la fuente; las tres alternativas gráficas (mover x_1 a 2, x_4 a 3 o x_2 a 3) quedan explicadas por sus estados y puntuaciones.

Como aclaración auxiliar, una búsqueda local guiada por conflictos puede atascarse o entrar en ciclos; una puntuación baja no prueba que desde allí se llegue a una solución. La heurística no es una estimación del coste óptimo h* de A*: cuenta violaciones. Tampoco deben contarse como «conflictos» las reinas amenazadas, pues una misma reina puede participar en varios pares.

4.2. Asignaciones parciales y conmutatividad

El estado inicial es la asignación vacía {}. El operador elige una variable todavía no asignada y un valor de su dominio, manteniendo la consistencia de la asignación parcial. La meta asigna las n variables y satisface todas las restricciones. La profundidad máxima es n porque cada ampliación fija una variable nueva; en una rama nunca se reasigna una variable, aunque al retroceder se prueben otras alternativas.

Si se generan sucesores eligiendo todas las variables posibles, en la raíz hay $n \cdot d$ posibilidades, después $(n-1) \cdot d$, y así hasta d. Sin podas, el número de hojas es $n! \cdot d^n$. Sin embargo, solo hay d^n asignaciones completas cuando todos los dominios tienen tamaño d. La diferencia n! corresponde a los órdenes de asignación: fijar A=1 y luego B=2 produce el mismo resultado que hacerlo al revés.

La solución es elegir una sola variable por nodo y ramificar únicamente sobre sus valores. Esa elección puede ser fija o depender de la asignación actual. No hay que explorar otros órdenes para conservar la completitud: toda solución debe asignar algún valor a la variable elegida. La conmutatividad se refiere a asignar variables distintas en una rama,

no a que puedan cambiarse valores ya fijados sin efecto.

5. Vuelta atrás y decisiones de búsqueda

5.1. Vuelta atrás cronológica

El chronological backtracking recorre en profundidad el espacio de asignaciones parciales. Al fallar una rama, vuelve a la última variable asignada que todavía tiene valores por probar. El algoritmo mantiene el invariante de que la asignación de entrada es consistente. Por eso, cuando está completa, puede devolverla como solución.

```
BT(CSP, σ):
  si σ es completa: devolver σ
  x ← elegir una variable no asignada
  para cada v del dominio de x, en el orden elegido:
    σ2 ← σ ∪ {x=v}
    si σ2 es consistente:
      resultado ← BT(CSP, σ2)
      si resultado ≠ fallo: devolver resultado
  devolver fallo
```

El dominio que se recorre es una lista local de alternativas: retirar un valor ya probado no significa borrarlo del problema original. Si se implementan asignaciones mutables, hay que deshacer x=v al salir de la rama. En la notación de la diapositiva 15, «σ′∈R» es una abreviatura de consistencia, no pertenencia literal de una asignación a un conjunto de predicados.

La completitud depende de que los dominios sean finitos y se prueben todos los valores que no hayan sido descartados justificadamente. Si existe una solución, la rama que toma sus valores nunca viola una restricción y no será podada por inconsistencia. El algoritmo acabará alcanzándola. Elegir otro orden cambia el trabajo y la primera solución, pero no esta garantía.

5.2. Árbol de cuatro reinas, reconstruido paso a paso

Usamos orden x_1, x_2, x_3, x_4 y columnas ascendentes. En la tabla, el prefijo (1,4) significa $\{x_1=1, x_2=4\}$. Las candidatas incompatibles se muestran para que el lector pueda reconstruir las cruces de la diapositiva 14; no se expanden como estados válidos.

Prefijo	Variable y pruebas	Resultado
()	$x_1=1$	Se abre la primera rama.
(1)	$x_2=1,2$ se rechazan; $x_2=3$	Misma columna o diagonal; 3 es compatible.
(1,3)	$x_3=1,2,3,4$ se rechazan	1 comparte columna; 2 ataca a fila 2; 3 comparte columna; 4 ataca a fila 2. Volver a x_2.
(1)	$x_2=4$	Se abre otra rama.
(1,4)	$x_3=1$ se rechaza; $x_3=2$	2 es compatible con ambas anteriores.

Prefijo	Variable y pruebas	Resultado
(1,4,2)	x_4=1,2,3,4 se rechazan	1 comparte columna; 2 comparte columna; 3 ataca a fila 3; 4 comparte columna (y ataca a fila 1).
(1,4)	x_3=3,4 se rechazan	3 ataca a fila 1; 4 comparte columna. Se agota x_1=1.
(2)	x_2=1,2,3 se rechazan; x_2=4	Se entra en la segunda rama de la raíz.
(2,4)	x_3=1	Primer valor compatible.
(2,4,1)	x_4=1,2 se rechazan; x_4=3	Se obtiene la primera solución.

El árbol dibujado en clase muestra además hermanos no recorridos después del éxito: x_3=2,3,4 bajo (2,4), x_4=4 bajo (2,4,1), y las ramas x_1=3,4. Dibujar un sucesor no significa que la ejecución de búsqueda de primera solución haya llegado a evaluarlo. Si se pidiesen todas las soluciones, se continuaría hasta encontrar también (3,1,4,2).

5.3. Heurísticas: qué variable y qué valor elegir

Decisión	Criterio	Motivo
Variable: MRV	Mínimos valores restantes: elegir el menor dominio viable actual.	Detectar pronto una variable sin posibilidades (fail first).
Variable: grado	Preferir la que condiciona más variables todavía no asignadas.	Una decisión difícil repercute en más lugares. Es útil para desempatar MRV.
Valor menos limitante	Preferir el que elimina menos opciones de las variables no asignadas.	Intentar una extensión prometedora (fail last).

No hay contradicción entre variable difícil y valor prometedor. Hay que asignar todas las variables, así que conviene descubrir cuanto antes una imposible; en cambio, basta un valor adecuado por variable. En Australia, SA tiene cinco vecinos, frente a ninguno de T. En cuatro reinas, tras x_1=2 y forward checking, los dominios son D_2={4}, D_3={1,3}, D_4={1,3,4}: MRV selecciona x_2. Si un ejercicio fija orden ascendente, esa instrucción prevalece sobre las heurísticas.

El número de restricciones se entiende según la representación. Con varias condiciones para una misma pareja suele ser útil combinarlas y contar vecinos no asignados. Un valor menos limitante requiere estimar o calcular eliminaciones; ese coste adicional puede compensarse con menos búsqueda. En caso de empate, debe explicitarse una regla reproducible.

5.4. Thrashing, salto atrás y nogoods

El thrashing repite fracasos causados por decisiones anteriores. Si $x_i=a$ es incompatible con todos los valores de x_k, cambiar variables intermedias que no solucionan ese conflicto lleva a descubrir el mismo fracaso muchas veces. El algoritmo sigue siendo correcto: el problema es la cantidad de trabajo, no que evaluar una restricción repetidamente cambie su resultado.

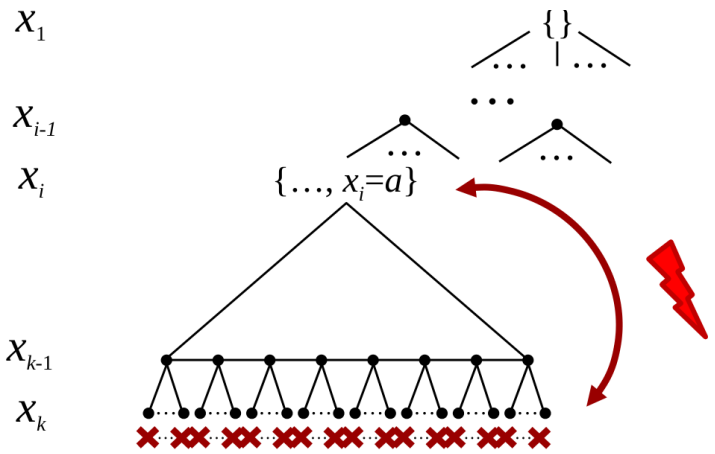


Figura 10. Subárbol repetido por thrashing, recortado de la diapositiva 17. El conflicto en x_k depende de una decisión anterior $x_i=a$.

El **backjumping** utiliza las causas del fallo para saltar decisiones irrelevantes. En una versión basada en conjuntos de conflicto se retrocede a la variable asignada más recientemente dentro del conjunto responsable, combinando la información de conflictos cuando corresponde. La frase docente «primera variable que causó el fallo» no autoriza a saltar arbitrariamente a la primera variable del árbol.

En el ejemplo de ocho reinas de la diapositiva 18, el prefijo $x_1=1, x_2=3, x_3=5, x_4=2$ ya impide todos los valores de x_6. Cambiar x_5 no resuelve esa causa. La tabla siguiente verifica qué reina anterior basta para excluir cada columna de la fila 6.

Columna de x_6	1	2	3	4	5	6	7	8
Reina que la excluye	x_1	x_3 o x_4	x_2	x_4	x_3	x_1	x_2	x_3

Se salta x_5 y se reconsidera x_4. La imagen animada superpone las posiciones antigua (columna 2) y alternativa (columna 7) de x_4; no representa dos reinas simultáneas en la misma fila. Tras cambiar x_4, las decisiones posteriores deben reconstruirse. El objetivo del salto es eliminar una causa demostrada del fallo, no garantizar que el siguiente intento sea solución.

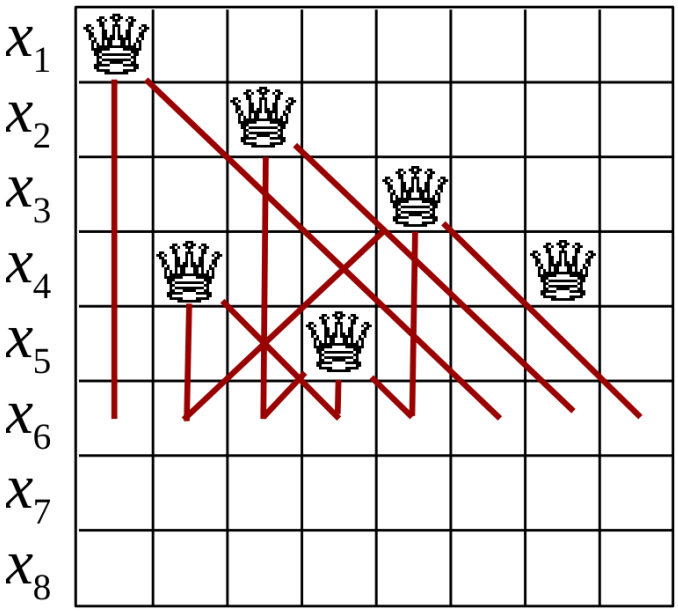


Figura 11. Recorte de la diapositiva 18: ataques a la fila 6 y posiciones de x_4 superpuestas por la animación. Deben leerse como alternativas sucesivas.

Un **nogood** es una asignación parcial que se ha demostrado que no puede extenderse a ninguna solución. Se recuerda para evitar futuras ramas que la contengan. En el ejemplo esquemático, $\{x_i=a\}$ puede registrarse como nogood solo si la imposibilidad es independiente del resto de decisiones. Si depende también de $x_j=b$, el nogood correcto debe incluir esa condición; generalizarlo indebidamente eliminaría soluciones.

Test 1.2(b) y 1.3(a). Ambas son falsas: thrashing supone repetición ineficiente, no resultados incorrectos; backjumping es una mejora del retroceso en búsqueda en profundidad, no una búsqueda en amplitud.

6. Propagación y comprobación hacia delante

6.1. Reducir dominios sin perder soluciones

Propagar restricciones consiste en eliminar de los dominios valores que no pueden participar en una solución bajo las decisiones actuales. Puede hacerse antes de buscar, como preprocesamiento, o después de cada asignación. El CSP reducido conserva las soluciones del CSP que se está filtrando. Si se ha fijado x=v, ese problema ya representa solamente la rama x=v del original.

Al colocar $x_1=1$ en cuatro reinas, se eliminan 1 y 2 de D_2, 1 y 3 de D_3, y 1 y 4 de D_4. Quedan $D_2=\{3,4\}$, $D_3=\{2,4\}$, $D_4=\{2,3\}$. Cada eliminación se justifica por columna o diagonal con la reina asignada. No se ha demostrado aún que las opciones restantes se puedan combinar entre sí.

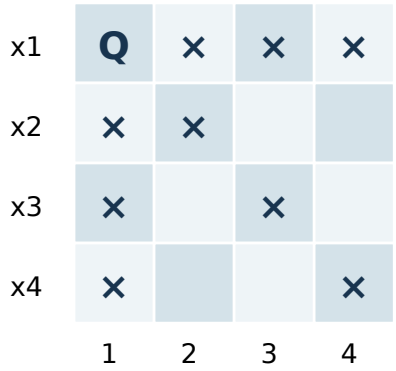


Figura 12. Primer filtrado tras $x_1=1$. Las cruces son valores eliminados, no reinas adicionales. Reconstrucción de la diapositiva 20.

6.2. Forward checking: mecanismo y restauración

La **comprobación hacia delante** (forward checking, FC) acompaña al backtracking. Tras asignar $x_i=v$, recorre cada vecino x_j no asignado y elimina de D_j los valores v' incompatibles con v. Si un dominio queda vacío, la rama fracasa inmediatamente. Basta filtrar desde la nueva asignación si el resto de decisiones ya se había propagado antes.

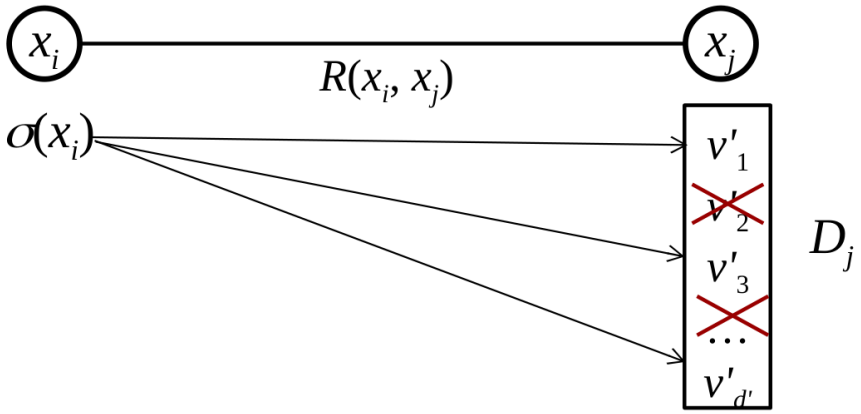


Figura 13. Mecanismo original de FC (diapositiva 21): un valor asignado se compara con los candidatos de una variable futura.

```
FC tras asignar x=v:
  para cada vecino y no asignado:
    para cada w de una copia de D[y]:
      si (v,w) viola la restricción entre x e y:
        quitar w de D[y] y registrar la eliminación
    si D[y] está vacío: devolver fallo
  devolver éxito
```

Al retroceder se restauran los dominios que existían antes de esa decisión. Se puede copiar el conjunto de dominios por rama o mantener una pila de eliminaciones. Restaurar siempre los dominios originales sería incorrecto si borra efectos de decisiones anteriores; no restaurar nada también sería incorrecto, porque mezclaría podas de ramas alternativas.

6.3. Cuatro reinas con FC: toda la secuencia

Decisión / prefijo	Dominios no asignados tras FC	Consecuencia
$x_1=1$	$D_2=\{3,4\}$; $D_3=\{2,4\}$; $D_4=\{2,3\}$	La rama continúa.
$x_1=1,x_2=3$	$D_3=\emptyset$; $D_4=\{2\}$	Fallo sin asignar x_3.
Restaurar y probar $x_2=4$	$D_3=\{2\}$; $D_4=\{3\}$	Todavía no detecta el conflicto entre x_3 y x_4.
$x_1=1,x_2=4,x_3=2$	$D_4=\emptyset$	Fallo; se agota la rama $x_1=1$.
Restaurar raíz; $x_1=2$	$D_2=\{4\}$; $D_3=\{1,3\}$; $D_4=\{1,3,4\}$	El siguiente x_2 solo puede valer 4.
$x_1=2,x_2=4$	$D_3=\{1\}$; $D_4=\{1,3\}$	Se fuerza el siguiente valor de x_3.
$x_1=2,x_2=4,x_3=1$	$D_4=\{3\}$	Se asigna $x_4=3$.
(2,4,1,3)	Todas las variables asignadas	Solución.

La tabla consolida los árboles y los tableros de las diapositivas 22 y 23. En (1,3), la fuente muestra el resultado de filtrar todos los vecinos aunque D_3 ya se haya vaciado; una implementación que se detenga en el primer dominio vacío no tiene que calcular $D_4=\{2\}$. La conclusión de fallo es la misma.

La limitación aparece en (1,4): $D_3=\{2\}$ y $D_4=\{3\}$ son no vacíos, pero esos valores se amenazan entre filas contiguas. FC solo ha comprobado las restricciones entre variables asignadas y futuras. Como x_3 y x_4 no están asignadas, no revisa su relación. Los dominios singleton pueden seguir perteneciendo a variables no asignadas: deducir un único candidato y efectuar una asignación son operaciones distintas.

La «transitividad» docente. La diapositiva 23 usa este término para indicar propagación de consecuencias a través de varias restricciones. No afirma que toda relación del CSP sea transitiva como propiedad matemática. La desigualdad $\neq$, por ejemplo, no lo es.

6.4. Familias y casas: modelado y FC (ejercicio 7)

Las variables A,B,C,D indican el número de casa de cada familia. $D_A=D_B=D_D=\{1,2,3,4\}$; $D_C=\{2,3\}$ porque C no vive en una esquina. D vive antes que B: $D<B$. B vive después de A: $A<B$. Entre B y C hay al menos otra casa: $|B-C|>1$. Ninguna familia comparte casa: todos-diferentes(A,B,C,D).

Para un grafo binario basta combinar las condiciones de cada pareja. A–B usa $A<B$; B–D usa $D<B$; B–C usa $|B-C|>1$; A–C, A–D y C–D usan desigualdad. Las tres primeras ya implican que sus extremos sean distintos. No hay que interpretar «al menos una otra casa» como distancia 1: entre las casas 2 y 3 no hay ninguna.

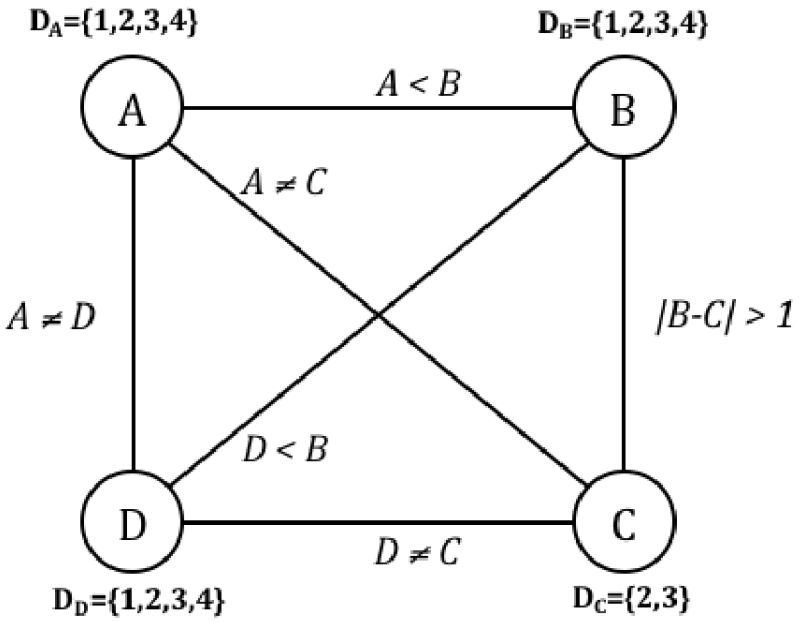


Figura 14. Grafo del ejercicio 7 (p. 8), con seis parejas relacionadas y C restringida a las casas interiores.

Paso en orden A,B,C,D	Dominios futuros	Explicación
A=1	B={2,3,4}; C={2,3}; D={2,3,4}	B debe superar 1; las restantes no pueden usarlo.
B=2	C=∅; D=∅	C=2 es igual y C=3 está al lado; D no puede ser menor que 2 con su dominio actual.
Restaurar; B=3	C=∅; D={2}	C=2 está al lado y C=3 coincide.
Restaurar; B=4	C={2}; D={2,3}	C=3 está al lado; D debe ser menor que 4.
C=2	D={3}	La casa 2 deja de estar disponible para D.
D=3	Solución (1,4,2,3)	Completa y cumple las seis relaciones.

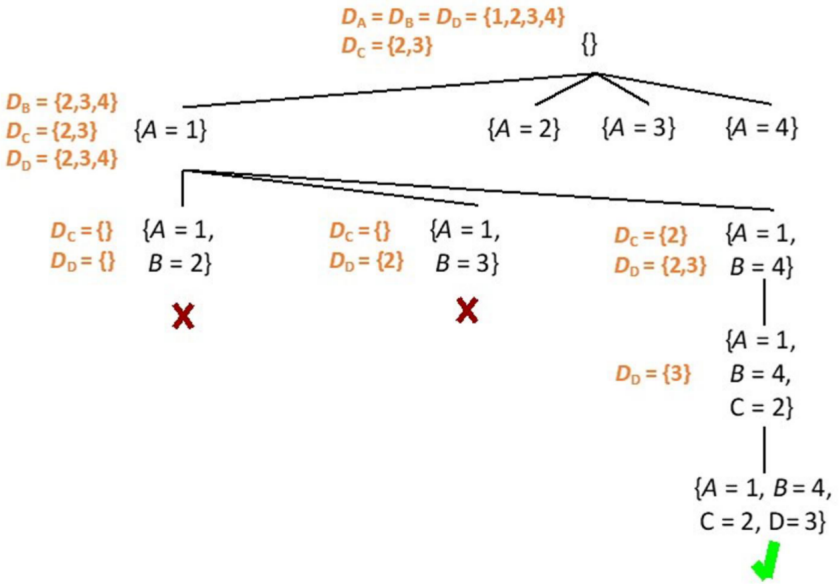


Figura 15. Árbol original de FC para las familias (p. 8). Los dominios de cada rama se detallan en la tabla anterior.

Al volver de B=2 o B=3 se recuperan los dominios posteriores a A=1, no los dominios iniciales. Con el orden pedido, la primera solución es {A=1,B=4,C=2,D=3}. Si se buscan todas las soluciones también aparece {A=3,B=4,C=2,D=1}. El apartado 8.3 muestra qué se descubre sin asignar mediante consistencia de arco.

7. Consistencia de arco y AC-3

7.1. Soportes y dirección de revisión

El arco $x_i \rightarrow x_j$ es consistente respecto a R si cada valor del dominio de x_i tiene al menos un valor compatible en el dominio de x_j. Ese valor compatible se llama **soporte**. La condición formal es:

$\forall v \in D_i$, $\exists v' \in D_j$ tal que $(v,v') \in R(x_i,x_j)$.
Orden de cuantificadores: cada v puede usar un soporte diferente. No se exige un único v′ compatible con todos los valores, ni que todas las parejas sean válidas.

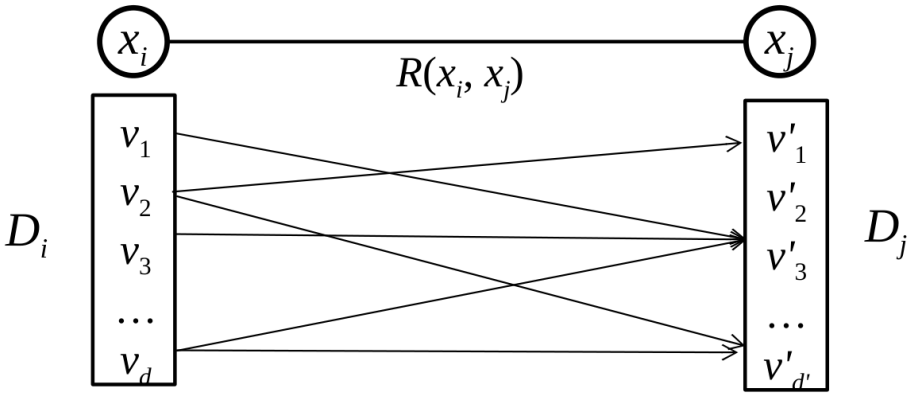


Figura 16. Soportes entre dominios, recortados de la diapositiva 24. Las flechas relacionan valores compatibles.

Una variable es arco-consistente si lo es respecto a todas las restricciones binarias en las que interviene. Una restricción binaria lo es si ambas direcciones son consistentes. El CSP es arco-consistente si lo son todas sus restricciones. Por ejemplo, con A<B, $D_A=\{1,2\}$, $D_B=\{2\}$, el arco B→A tiene soporte (B=2,A=1), pero A→B falla para A=2.

Al revisar $x_i \to x_j$ se eliminan valores solo de D_i. Para revisar el sentido contrario se intercambia el papel de los dominios, respetando el significado del predicado. En A<B, revisar B→A sigue buscando a<b; no se transforma en B<A.

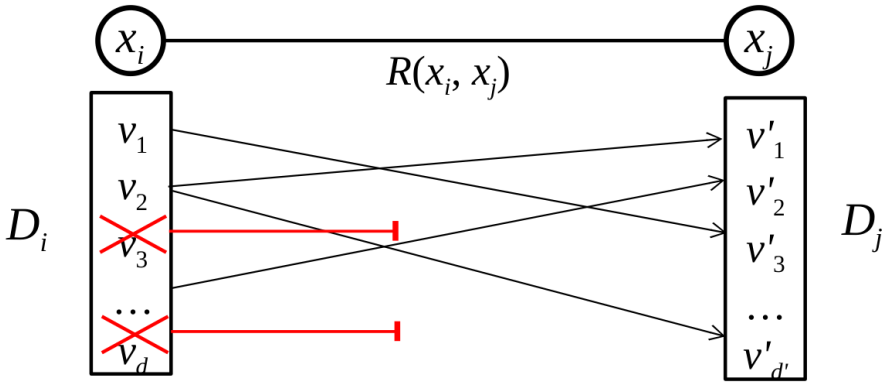


Figura 17. Valores sin soporte eliminados de D_i, recorte de la diapositiva 25. Una sola pareja incompatible no basta para eliminar un candidato.

La poda es segura: si v no tiene ningún soporte en D_j, una supuesta solución con $x_i=v$ necesitaría asignar a x_j algún valor de D_j que cumpliese R. Eso contradice la ausencia de soporte. Por tanto, ninguna solución usa v. Aplicando el argumento a cada eliminación se conserva el conjunto de soluciones del CSP filtrado.

Test 1.2. (a) es verdadera: una relación contenida en $D_i \times D_j$ es binaria. (c) es falsa: con A={1,2,4,5,6,12}, B={blue,green,orange} y A=|B|, las longitudes son 4,5,6. A=1,2,12 carecen de soporte. Tras eliminarlos quedan A={4,5,6} y las tres palabras. (d) es falsa: un dominio singleton no prueba inconsistencia; un dominio vacío sí.

7.2. Por qué hay que repetir revisiones

Una revisión puede borrar el último soporte que utilizaba un valor de otra variable. Si al revisar $x_i \to x_j$ cambia D_i, hay que reconsiderar los arcos entrantes $x_k \to x_i$. Revisar solo $x_i \to x_k$ atendería al dominio equivocado: los candidatos que han perdido soportes están en D_k.

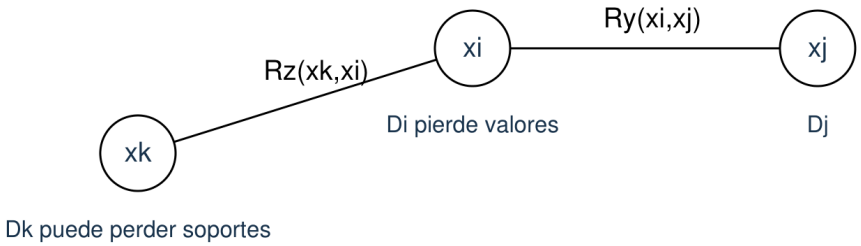


Figura 18. Reconstrucción de la diapositiva 27: al reducir D_i, x_k puede perder soportes y debe revisarse $x_k \to x_i$.

Una cola llamada abierta conserva los arcos pendientes. Inicialmente contiene ambos sentidos de todas las restricciones binarias. Cada iteración extrae un arco, revisa su dominio de origen y, si lo reduce, añade los arcos que pueden haber quedado inconsistentes. Las revisiones sin cambios no añaden trabajo. Una cola sin duplicados evita repeticiones simultáneas innecesarias.

7.3. Pseudocódigo operativo

```
AC-3(CSP):
  si existe un dominio vacío: devolver fallo
  abierta ← todos los arcos dirigidos
  mientras abierta no esté vacía:
    (xi,xj) ← extraer un arco de abierta
    si REVISAR(xi,xj):
      si D[xi] está vacío: devolver fallo
      para cada vecino xk de xi:
        añadir (xk,xi) a abierta si no está ya
  devolver CSP con los dominios reducidos

REVISAR(xi,xj):
  borrado ← falso
  para cada v de una copia de D[xi]:
    si no existe w en D[xj] compatible con v:
      quitar v de D[xi]
      borrado ← verdadero
  devolver borrado
```

Precisiones sobre la diapositiva 34. «primero(abierta)» debe retirar el elemento o acompañarse de una eliminación explícita; consultar siempre el primero sin retirarlo no permite terminar. Se ha añadido la detección inmediata de dominio vacío y la devolución del CSP reducido. La versión docente reintroduce todos los vecinos, incluido x_j: es correcta aunque puede efectuar revisiones redundantes. La variante habitual puede omitir x_j cuando las restricciones por pareja están combinadas.

Termina porque los dominios son finitos y solo disminuyen. Hay a lo sumo n·d eliminaciones; cada eliminación puede añadir una cantidad finita de arcos. Las extracciones que no cambian dominios consumen la cola. Al terminar sin fallo y sin arcos pendientes, todos los soportes que podrían haber desaparecido se han revisado.

7.4. La secuencia A, B, C de las diapositivas

Dominios iniciales: $D_A=\{2,3,4,5\}$, $D_B=\{2,3,4,5\}$, $D_C=\{1,2,3\}$. R_1 exige $A=B-1$; R_2 exige $C=(B+1)$ div 2, con división entera; R_3 exige $A>C$. En R_2, $B=2$ corresponde a $C=1$, $B=3$ a $C=2$, $B=4$ a $C=2$ y $B=5$ a $C=3$.

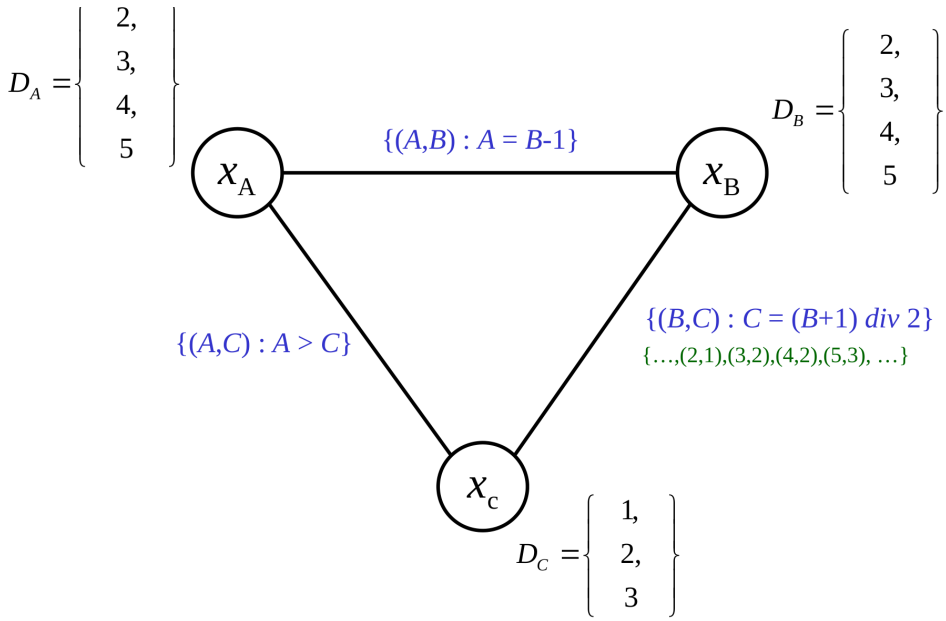


Figura 19. Grafo y dominios iniciales de la diapositiva 28. Los estados sucesivos de las diapositivas 29–33 se reconstruyen en la tabla.

Evaluación bilateral	D_A	D_B	D_C	Valores eliminados
Inicial	{2,3,4,5}	{2,3,4,5}	{1,2,3}	—
R_1	{2,3,4}	{3,4,5}	{1,2,3}	A=5 exigiría B=6; B=2 exigiría A=1.
R_2	{2,3,4}	{3,4,5}	{2,3}	C=1 ya no tiene B compatible.
R_3	{3,4}	{3,4,5}	{2,3}	A=2 no es mayor que ningún C.
R_1 de nuevo	{3,4}	{4,5}	{2,3}	B=3 pierde su soporte A=2.
R_2 y R_3	{3,4}	{4,5}	{2,3}	No hay nuevas eliminaciones.

Quedan D_A={3,4}, D_B={4,5}, D_C={2,3}. En R_1 los soportes son (3,4) y (4,5); en R_2, (4,2) y (5,3); en R_3, (3,2),(4,2),(4,3). Todos los valores tienen soporte en ambos sentidos de sus restricciones. Esto no significa que cualquier combinación del producto de dominios sea solución: (3,5,2), por ejemplo, viola A=B−1. Las soluciones completas son (3,4,2) y (4,5,3).

La diapositiva 35 detalla el comienzo mediante arcos dirigidos. Si abierta empieza [A→B,B→A,B→C,C→B,A→C,C→A], revisar A→B borra A=5; deben reconsiderarse B→A y C→A, que ya estaban pendientes. Revisar B→A borra B=2; se reintroduce A→B y se mantiene C→B pendiente. El resto continúa hasta alcanzar el mismo cierre. Esta traza explica las flechas y la reinserción del dibujo sin confundir una revisión dirigida con la fila bilateral de la tabla.

7.5. Garantías y complejidad

AC-3 conserva las soluciones y, si vacía un dominio, demuestra que el CSP filtrado no tiene solución. El recíproco es falso: puede terminar sin dominios vacíos en un problema imposible. En una rama de búsqueda, «no tiene solución» se refiere a esa rama, no necesariamente al CSP antes de las decisiones.

Sea n el número de variables, d el tamaño máximo de un dominio y e el número de arcos dirigidos. En un grafo binario simple hay como máximo $n(n-1)$ arcos. Una revisión puede comparar d valores de origen con d valores de destino: $O(d^2)$. Cada arco se reconsidera $O(d)$ veces por reducciones del dominio que aporta sus soportes, además de su inserción inicial. El tiempo es $O(e \cdot d^3)$, que en el peor caso denso se escribe $O(n^2 \cdot d^3)$, como en la diapositiva 36.

Esta cota supone que comprobar una pareja permitida cuesta $O(1)$, una cola que gestiona adecuadamente los pendientes y restricciones de la misma pareja combinadas. Si hay muchas restricciones independientes por pareja, deben contarse en e o en el coste de comprobarlas. La frase docente «a lo sumo d veces» se entiende asintóticamente; una contabilización literal puede incluir la entrada inicial además de las reinserciones.

La poda cuesta trabajo, pero puede evitar una cantidad mucho mayor de búsqueda. No garantiza que cualquier instancia sea más rápida: depende del tamaño de los dominios, de las restricciones y de lo que se elimine. La comprobación de extracción y reinserción se contrastó con la implementación oficial de AIMA; la explicación general de la cola y la cota $O(ed^3)$, con el texto docente CS188 de Berkeley (referencias al final).

8. Ejecutar consistencia de arco en problemas nuevos

8.1. Cuatro reinas y ausencia de soportes

Después de $x_1=1$ y el primer filtrado quedan $D_2=\{3,4\}$, $D_3=\{2,4\}$, $D_4=\{2,3\}$. Para $x_3=2$, ni $x_4=2$ ni $x_4=3$ son compatibles: comparten columna o diagonal. Se elimina 2 de D_3. Para $x_3=4$, ni $x_2=3$ ni $x_2=4$ son compatibles: diagonal o columna. Se elimina 4. D_3 se vacía; la rama $x_1=1$ es imposible sin necesidad de asignar x_2.

Es posible llegar al mismo fallo en otro orden. El solucionario trata R'_4 entre x_2 y x_3 antes de R'_6: deja $D_2=\{4\}$ y $D_3=\{2\}$; al tratar R'_6 no hay ninguna pareja válida con $D_4=\{2,3\}$. Se vacía D_3 y, si se continúa revisando el sentido contrario, también D_4. No hace falta vaciar todos los dominios para concluir el fallo.

8.2. Intervalos naturales y desigualdades estrictas (ejercicio 6)

Los dominios iniciales son $A\in[1,50]$, $B\in[20,40]$, $C\in[1,30]$, $D\in[1,30]$, donde [l,u] contiene todos los números naturales entre ambos extremos, incluidos. Las restricciones son R_1: $A\leq B$; R_2: $B+7<C$; R_3: $D\neq B$; R_4: $D>A$; R_5: $D+10<C$. Se evalúa cada restricción en ambos sentidos y en orden R_1,R_2,R_3,R_4,R_5, repitiendo hasta no cambiar nada.

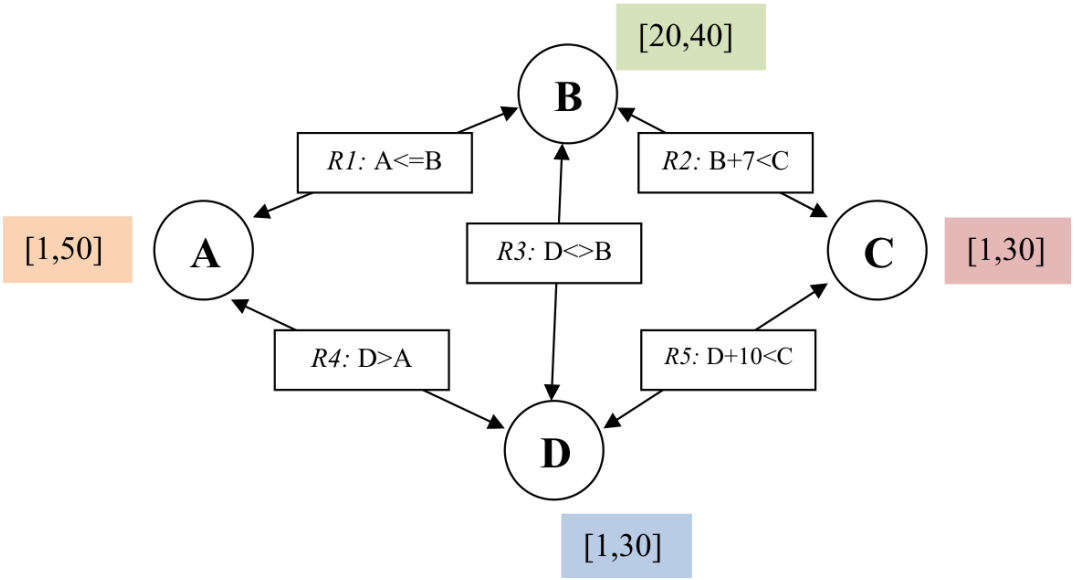


Figura 20. Grafo del ejercicio 6 (p. 6), con las cinco relaciones. No hay relación directa A–C.

R_1 elimina A>40 porque ningún B inicial puede soportarlo. R_2 usa que B≥20 y C≤30: debe cumplirse C≥28 y B≤22. El carácter estricto importa: B=23 exigiría C>30; C=27 no supera 20+7. R_4 reduce A≤29 y D≥2. R_5 obliga D≤19, pues D+10<30 implica D<20.

Vuelta / R	D_A	D_B	D_C	D_D
Inicio	[1,50]	[20,40]	[1,30]	[1,30]
1 / R1	[1,40]	[20,40]	[1,30]	[1,30]
1 / R2	[1,40]	[20,22]	[28,30]	[1,30]
1 / R3	[1,40]	[20,22]	[28,30]	[1,30]
1 / R4	[1,29]	[20,22]	[28,30]	[2,30]
1 / R5	[1,29]	[20,22]	[28,30]	[2,19]
2 / R1	[1,22]	[20,22]	[28,30]	[2,19]
2 / R2	[1,22]	[20,22]	[28,30]	[2,19]
2 / R3	[1,22]	[20,22]	[28,30]	[2,19]
2 / R4	[1,18]	[20,22]	[28,30]	[2,19]
2 / R5	[1,18]	[20,22]	[28,30]	[2,19]
3 / R1	[1,18]	[20,22]	[28,30]	[2,19]
3 / R2	[1,18]	[20,22]	[28,30]	[2,19]
3 / R3	[1,18]	[20,22]	[28,30]	[2,19]
3 / R4	[1,18]	[20,22]	[28,30]	[2,19]
3 / R5	[1,18]	[20,22]	[28,30]	[2,19]

En la segunda vuelta, R_1 transmite la reducción de B y deja $A \leq 22$. Después R_4 transmite $D \leq 19$ y deja $A \leq 18$. La tercera vuelta completa no cambia ningún dominio y certifica el punto fijo. R_3 no poda: que D contenga 20,21,22 inicialmente no obliga a borrarlos, porque cada uno puede elegir otro valor de B como soporte. Una desigualdad elimina un valor del vecino cuando su dominio queda fijado únicamente a ese mismo valor, no por la mera intersección de dominios.

Resultado: $A \in [1,18]$, $B \in [20,22]$, $C \in [28,30]$, $D \in [2,19]$. La combinación $(A,B,C,D)=(1,20,28,2)$ satisface las cinco restricciones y demuestra que esta instancia tiene solución. Los dominios finales no autorizan todas sus combinaciones: $A=18, D=2$ viola $D>A$. La tabla completa coincide con el solucionario y se ha recalculado por soportes discretos.

8.3. Familias: propagación sin asignaciones

Partimos del modelo de §6.4, no de los dominios de una rama FC. Se puede alcanzar el cierre mediante el siguiente orden de restricciones, el mismo que usa el solucionario. Cada fila indica eliminaciones justificadas por ausencia de soporte.

Restricción	Eliminaciones	Justificación
$A<B$	B: quitar 1; A: quitar 4	No existe A menor que 1 ni B mayor que 4.
$\lvert B-C\rvert>1$	B: quitar 2,3; C: quitar 3	C solo tiene 2,3. Solo B=4 tiene soporte, y únicamente con C=2.
$D<B$	D: quitar 4	$B=\{4\}$; D debe ser menor.
$C \neq D$	D: quitar 2	$C=\{2\}$.
$A \neq C$	A: quitar 2	$C=\{2\}$.
Todas las demás revisiones	Sin nuevas eliminaciones	Se comprueba $A \neq D$ y ambas direcciones de cada relación.

Queda $A=\{1,3\}$, $B=\{4\}$, $C=\{2\}$, $D=\{1,3\}$. $A \neq D$ tiene soportes cruzados: A=1 con D=3, y A=3 con D=1. Las dos soluciones son (1,4,2,3) y (3,4,2,1). En este caso concreto todos los valores restantes aparecen en alguna solución, pero eso no es una garantía general de consistencia de arco.

9. Límites de la propagación y k-consistencia

9.1. Arco-consistente no significa resoluble

Consideremos A,B,C con dominios {1,2} y las tres restricciones $A \neq B$, $B \neq C$ y $A \neq C$. Cada valor de cada variable tiene soporte en cada vecino: a 1 le sirve 2 y a 2 le sirve 1. Por eso el problema ya es arco-consistente y AC-3 no elimina nada.

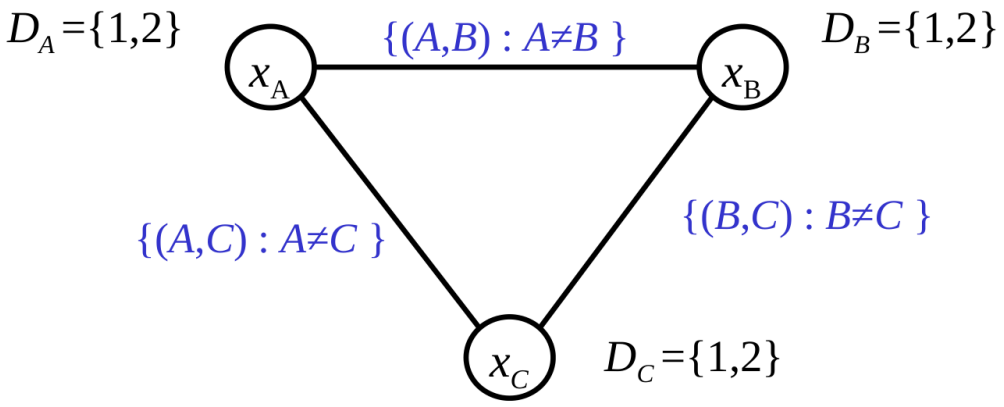


Figura 21. Contraejemplo original de la diapositiva 37: tres variables, dos valores y desigualdad entre cada pareja.

Sin embargo, A=1 obliga B=2, y entonces C tendría que ser distinto de 1 y de 2 simultáneamente. No puede. El caso A=2 es simétrico. Por tanto, no existe solución. La causa es que los soportes individuales pueden no ser compatibles entre sí. No basta con comprobar aristas por separado para garantizar una asignación conjunta.

9.2. k-consistencia y consistencia fuerte

Un CSP es **k-consistente** si cualquier asignación consistente a k−1 variables puede extenderse consistentemente a cualquier otra variable elegida. No significa que exista alguna extensión para alguna variable: la condición es universal sobre las asignaciones parciales consistentes y sobre la variable restante.

Nivel	Qué exige
1-consistencia	Cada variable puede recibir un valor compatible con sus restricciones unarias; los dominios se filtran respecto a ellas. Para construir una solución se necesita que sean no vacíos.
2-consistencia	Cada asignación consistente a una variable se extiende a otra. En redes binarias con las restricciones unarias ya incorporadas corresponde a consistencia de arco.
3-consistencia	Cada asignación consistente a dos variables se extiende a una tercera cualquiera. En la terminología del tema, consistencia de camino.
Fuerte k-consistencia	Se cumplen conjuntamente los niveles 1,2,...,k.

El triángulo anterior no es 3-consistente: A=1,B=2 es una asignación consistente de dos variables que no se puede extender a C. La consistencia de camino puede exigir eliminar parejas permitidas de relaciones, no solo valores de dominios. Este detalle explica por qué es una condición diferente y más costosa que revisar soportes individuales.

Si una red de n variables es fuertemente n-consistente y se parte de dominios no vacíos, se puede construir una solución incrementalmente. Se elige un valor válido para la primera variable. La 2-consistencia permite extenderlo a una segunda; la 3-consistencia extiende esa asignación a una tercera; y se repite hasta n. Cada paso conserva la consistencia y, al final, la asignación es completa. La palabra «fuertemente» importa: n-consistencia por sí sola no garantiza que existan las asignaciones parciales necesarias

para iniciar esa construcción.

Precisión sobre el vacío. Algunas definiciones formales de consistencia de nodo solo exigen que todos los valores presentes cumplan las restricciones unarias, lo que se cumple vacíamente si el dominio es vacío. Por eso el enunciado de existencia debe explicitar dominios no vacíos; una red con un dominio vacío nunca tiene solución.

10. MAC: búsqueda y propagación combinadas

10.1. Mantener consistencia de arcos

MAC significa Maintaining Arc Consistency. El material lo denomina también Full Look-Ahead (FLA). Realiza backtracking y, tras aumentar la asignación, aplica consistencia de arcos al CSP de esa rama. Así permite que las consecuencias pasen entre variables todavía no asignadas, que es precisamente lo que FC no hace.

La operación σ′(CSP) de la diapositiva 39 significa restringir a {v} el dominio de cada variable fijada como x=v. Después se calcula la consistencia de arco. El CSP reducido es equivalente al problema condicionado por σ′, no al CSP original sin decisiones. Si se vacía un dominio, se descarta esa rama y se restauran los dominios del nodo padre.

```
MAC(CSP, σ):
  si hay algún dominio vacío: devolver fallo
  si σ es completa: devolver σ
  x ← elegir variable no asignada
  para cada v del dominio actual de x:
    σ2 ← σ ∪ {x=v}
    si σ2 es consistente:
      hijo ← copia del CSP actual
      D_hijo[x] ← {v}
      reducido ← AC-3(hijo)
      si reducido ≠ fallo:
        resultado ← MAC(reducido, σ2)
        si resultado ≠ fallo: devolver resultado
  devolver fallo
```

Este pseudocódigo explicita las copias y el control de dominio vacío omitidos en la presentación. Las copias son una opción pedagógica: una implementación eficiente puede registrar y deshacer podas. Si todos los dominios son singleton y el CSP binario es arco-consistente, sus valores forman una solución; aun así, el pseudocódigo docente puede continuar asignando explícitamente las variables restantes.

10.2. Cuatro reinas con MAC (ejercicio 5)

Se eligen filas y columnas ascendentes. En la raíz no se elimina ningún valor por arco-consistencia. Al probar $x_1=1$ se obtiene el fallo explicado en §8.1. La tabla reproduce el orden bilateral del solucionario; las celdas muestran el estado completo, evitando que un hueco se interprete como dominio vacío.

Evaluación	D_1	D_2	D_3	D_4
Fijar $x_1=1$	{1}	{1,2,3,4}	{1,2,3,4}	{1,2,3,4}
R'_1	{1}	{3,4}	{1,2,3,4}	{1,2,3,4}

Evaluación	D_1	D_2	D_3	D_4
R'_2	{1}	{3,4}	{2,4}	{1,2,3,4}
R'_3	{1}	{3,4}	{2,4}	{2,3}
R'_4	{1}	{4}	{2}	{2,3}
R'_6	{1}	{4}	∅	∅ si se revisan ambos sentidos

Al probar $x_1=2$, el filtrado directo da $D_2=\{4\}$, $D_3=\{1,3\}$, $D_4=\{1,3,4\}$. R'_4 elimina $x_3=3$ porque ataca a $x_2=4$. La relación x_2–x_4 elimina 4, y con $x_3=1$ la relación x_3–x_4 elimina 1. El cierre deja $D_2=\{4\}$, $D_3=\{1\}$, $D_4=\{3\}$. La búsqueda asigna esos valores y obtiene (2,4,1,3).

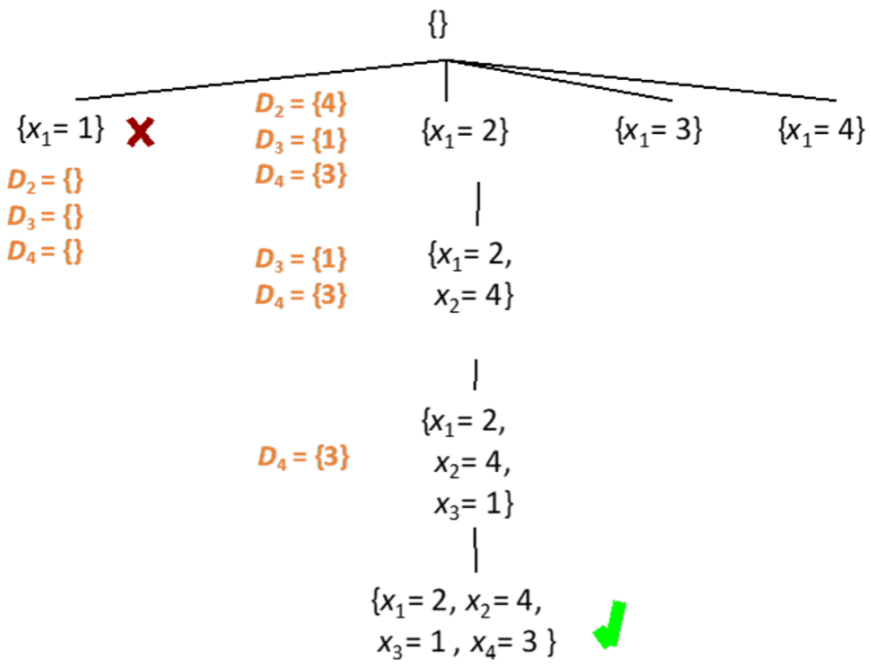


Figura 22. Árbol MAC del solucionario (p. 5). El fracaso bajo $x_1=1$ se detecta antes de asignar x_2; bajo $x_1=2$ todos los dominios quedan unitarios.

En la rama fracasada, el dibujo muestra vacíos todos los dominios futuros. Una propagación que continúe después del primer vacío puede extender la contradicción por el grafo, llegando incluso a vaciar el dominio de la variable fijada. Una implementación con parada temprana puede mostrar solo el primer vacío. Estas trazas son compatibles: todas prueban que esa rama no tiene solución.

10.3. Comparar qué detecta cada método

Método	Momento de detección en $x_1=1$	Qué comprueba
Backtracking simple	Prueba variables posteriores hasta agotar ramas.	Restricciones con todos sus extremos ya asignados.
Backtracking + FC	Falla en (1,3) o en (1,4,2).	Nueva variable asignada frente a variables futuras.
MAC	Falla ya tras $x_1=1$.	Soportes entre variables, incluidas las futuras, hasta cierre.
AC-3 aislado	Puede podar o demostrar un fallo; no enumera decisiones.	Solo propagación sobre el CSP recibido.

Las heurísticas de variable y valor son compatibles con BT, FC y MAC. FC y MAC no son A*: no se ordena una frontera por g+h. MAC mantiene la completitud de backtracking con dominios finitos porque la propagación solo elimina valores que no pueden formar parte de una solución de la rama.

Test 1.3. Son verdaderas (b) y (d): MAC combina vuelta atrás cronológica con consistencia de arcos e intercala satisfacción y propagación. (c) no es la definición docente: FC es un filtrado más débil, que MAC sustituye por mantenimiento de consistencia de arcos; no son dos algoritmos de búsqueda independientes que se sumen. (a) y (e) son falsas: ni backjumping es amplitud ni FC realiza A* con asignaciones totales.

11. Modelar y ejecutar una agenda de visitas

11.1. Restricciones del representante (ejercicio 8)

Las variables son Repsol, Telefónica, ACS, Endesa, Iberdrola y UFenosa. Representan horas enteras de visita. Todos los dominios son $\{9,10,\ldots,19\}$, salvo ACS, cuyo dominio es $\{15,16,17\}$. Se visitan clientes a horas diferentes; ACS y UFenosa deben visitarse antes de Endesa; entre Repsol y cualquier otro cliente debe haber al menos dos horas.

Interpretación del horario. Se sigue el modelo explícito del solucionario: incluye 19 y 17 como horas posibles y exige $|\text{hora(Repsol)}-\text{hora(otro)}|\geq 2$. El enunciado no especifica duración de cada visita ni suma una hora de atención al desplazamiento. Si las visitas durasen una hora y después se necesitasen dos horas de viaje, la separación de inicios sería ≥3. Esa variante daría otro CSP y no debe introducirse silenciosamente.

Hay una restricción combinada para cada una de las 15 parejas. Para ACS–Endesa y UFenosa–Endesa se usa precedencia estricta (<), que ya implica desigualdad. Para las cinco parejas que incluyen Repsol se usa separación absoluta ≥2, que también implica desigualdad. Para las ocho parejas restantes basta ≠. Si se invierte el orden con que se escribe una pareja de precedencia, debe invertirse la comparación.

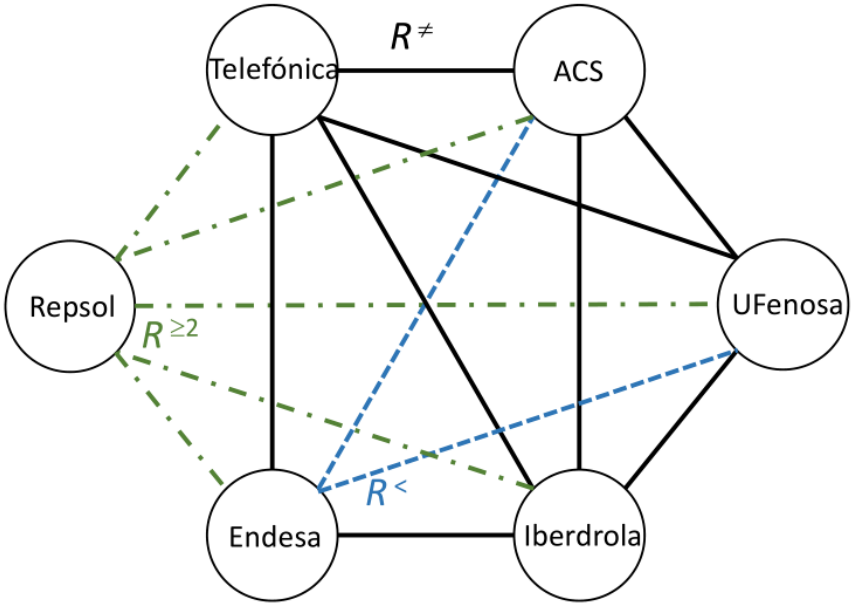


Figura 23. Grafo original del ejercicio 8 (p. 10). Los tres tipos de línea expresan desigualdad, precedencia y separación de dos horas; las reglas se detallan también en texto.

Una formulación alternativa equivalente conserva todos-diferentes sobre las seis variables y añade las dos precedencias y las cinco separaciones. Ambas tienen las mismas soluciones. El grafo simple agrupa condiciones por pareja; no significa que haya que contar varias aristas para cada desigualdad redundante.

11.2. Backtracking en el orden exigido

Se prueban variables en el orden Repsol, Telefónica, ACS, Endesa, Iberdrola, UFenosa, y valores de menor a mayor. Se comprueban las restricciones que ya tienen ambos extremos asignados. No se aplica FC ni MAC porque el apartado pide backtracking simple.

Variable	Pruebas relevantes	Valor aceptado
Repsol	No hay decisiones anteriores.	9
Telefónica	9 coincide; 10 queda a una hora de Repsol.	11
ACS	El dominio empieza en 15; cumple separación y distinción.	15
Endesa	9–15 fallan: 9 y 10 por Repsol; 11 por Telefónica; 12–15 no son posteriores a ACS.	16
Iberdrola	9 y 10 fallan por Repsol; 11 coincide con Telefónica.	12
UFenosa	9 y 10 fallan por Repsol; 11 y 12 coinciden con visitas previas; 13 es anterior a Endesa.	13

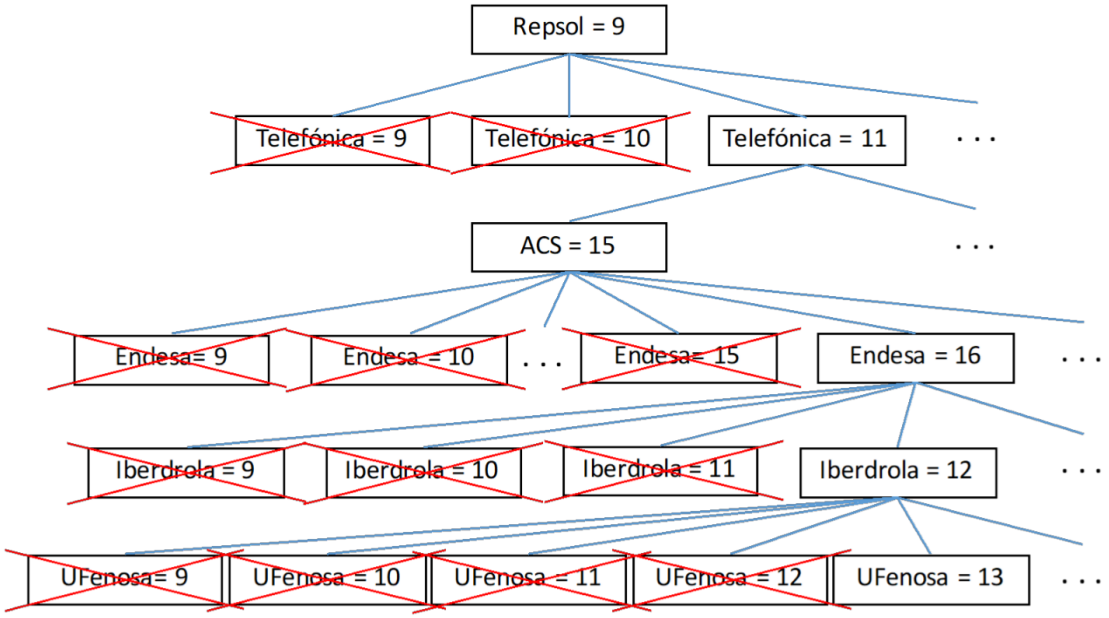


Figura 24. Árbol original de primeras pruebas del ejercicio 8 (p. 10), integrado como apoyo a la traza razonada.

La primera solución en el orden de variables es (9,11,15,16,12,13). En orden cronológico: Repsol a las 9, Telefónica a las 11, Iberdrola a las 12, UFenosa a las 13, ACS a las 15 y Endesa a las 16. Las seis horas son distintas; $|9-11|=2$ y todas las demás separaciones con Repsol son mayores; $15<16$ y $13<16$. Se satisfacen las 15 relaciones del modelo.

Obsérvese que Endesa puede asignarse antes de UFenosa en el árbol aunque se visite después. El orden de búsqueda no es el orden temporal del plan. La restricción UFenosa<Endesa se comprueba al asignar UFenosa. En este caso no hace falta retroceder de una asignación aceptada, aunque sí se rechazan muchos candidatos inmediatamente. Con otros horarios o un dominio distinto de ACS podrían aparecer retrocesos reales.

12. Extensiones y preparación de la evaluación

12.1. Qué cambia en CSP más avanzados

La diapositiva 40 enumera líneas avanzadas sin desarrollar sus algoritmos. Las siguientes aclaraciones delimitan su significado; no añaden técnicas obligatorias que no aparecen en los archivos.

Extensión del temario	Significado y cambio respecto al modelo básico
Algoritmos y heurísticas más sofisticados	Mejoran la detección de conflictos, la propagación o la elección de variables y valores. La prioridad sigue siendo conservar soluciones cuando la poda pretende ser exacta.
Dominios infinitos y continuos	Las variables pueden recorrer enteros sin cota o números reales. Ya no se pueden enumerar necesariamente todos sus valores; los argumentos de finitud de este capítulo dejan de aplicarse sin más.

Extensión del temario	Significado y cambio respecto al modelo básico
Estructuración jerárquica	Agrupa partes del problema y explota sus dependencias para organizar la resolución. La estructura importa además de la cantidad de variables.
COP: optimización de restricciones	Entre asignaciones factibles se prefiere la que optimiza un objetivo o un conjunto de preferencias. Encontrar cualquier solución puede ser insuficiente.
DCSP y DCOP	La información y las decisiones se distribuyen entre agentes. Además de restricciones y objetivos, hay que coordinar decisiones mediante comunicación.
Relajación de restricciones	Se debilitan o retiran condiciones para obtener soluciones cuando el modelo original no es viable o para manejar preferencias. En general cambia el conjunto de soluciones: no es la poda equivalente de AC-3.

12.2. Guion para resolver un enunciado nuevo

Modelar. Define qué significa cada variable y sus unidades; fija dominios y pistas; traduce cada frase del enunciado a una restricción; verifica extremos incluidos, desigualdades estrictas y relaciones implícitas. Comprueba que el modelo no permite compartir una casa u hora cuando está prohibido, ni prohíbe repetir colores donde sí se permite.

Representar. Dibuja todas las variables, incluidas las aisladas. Etiqueta aristas o hiperarcos con el predicado correcto. Si conviertes a binario, escribe el orden de componentes de cada tupla auxiliar y sus igualdades de proyección. Verifica cada tupla sustituyéndola en la fórmula original.

Ejecutar. Respeta el orden de variables, valores y restricciones que pida el ejercicio. Distingue candidatas rechazadas de estados expandidos, asignaciones de dominios singleton, revisiones dirigidas de bilaterales. En FC y MAC guarda los dominios del padre antes de explorar cada hijo. En AC-3 registra la ausencia de soporte y repite las revisiones necesarias hasta el cierre o el primer vacío.

Justificar. Un fallo necesita una restricción violada o un dominio vacío. Una solución necesita todas las variables asignadas y todas las restricciones satisfechas. El final de AC-3 sin vacíos solo certifica consistencia de arco; para afirmar existencia, muestra una solución o una garantía adicional aplicable. Cuando se piden todas las soluciones, no termines al encontrar la primera.

12.3. Distinciones que suelen decidir una pregunta teórica

Afirmación	Evaluación razonada
«Parcial consistente» implica «extensible».	Falsa: (x_1=1,x_2=4) de cuatro reinas es localmente consistente y no tiene extensión completa.
«Completa» implica «solución».	Falsa: también debe cumplir todas las restricciones.
Un singleton implica fallo.	Falsa: representa una única opción. El vacío implica fallo.
Un CSP arco-consistente tiene solución.	Falsa: triángulo de desigualdades con dominios {1,2}.
Cambiar el orden destruye la completitud de BT.	Falsa si se siguen explorando todas las alternativas válidas en dominios finitos. Puede cambiar enormemente el tiempo.
Borrar un valor por no tener soporte es seguro.	Verdadera: una solución que lo usara requeriría ese soporte inexistente.
AC-3 equivale a buscar una solución.	Falsa: propaga; MAC añade las decisiones y el retroceso.
Codificar a binario mantiene necesariamente el mismo coste.	Falsa: conserva soluciones proyectadas, pero puede aumentar mucho los dominios auxiliares.

13. Fuentes y alcance de las reconstrucciones

Fuentes docentes principales. «00 Diapositivas-Tema6_con_animaciones.ppsx», 40 diapositivas, y «02 Ejercicios-Tema6-Solución.pdf», 10 páginas. El manual toma de ellas el temario, las relaciones, los datos y las convenciones de evaluación. «Texto pegado(5).txt» establece las instrucciones de elaboración.

Consulta complementaria puntual. AIMA, código oficial csp.py: aima.cs.berkeley.edu/python/csp.html. UC Berkeley, CS188, «2.3 Filtering»: inst.eecs.berkeley.edu/~cs188/textbook/csp/filtering.html. Se usaron para contrastar la extracción y reinserción de arcos y la cota de AC-3, sin sustituir las convenciones docentes.

Reconstrucciones. Se extrajeron el texto, los objetos y las secuencias de animación del PPSX y se revisó su representación estática. Las tablas de BT, FC, AC-3 y MAC explicitan estados necesarios, conservando los datos. Las repeticiones de los índices del módulo y los elementos decorativos se consolidan; los diagramas relevantes se recortan individualmente o se reconstruyen con tablas y tableros legibles.

Límite de la verificación de animaciones. Se ha contrastado el orden de objetos y los estados algorítmicos; no se ha reproducido la temporización visual en PowerPoint. En particular, la superposición de posiciones en el ejemplo de salto atrás se interpreta como estados sucesivos, no como un tablero único. El informe separado documenta las correcciones y las comprobaciones realizadas.